



UNIVERSIDADE DE FORTALEZA
VICE REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Anna Correia - 2325494

Lais Barbosa - 2322637

Leticia Lucena - 2322682

Lorena Lin - 2322680

Petrus Galvão - 2322674

DOCUMENTAÇÃO TÉCNICA

Dental Connect

FORTALEZA

2025

Anna Correia - 2325494
Lais Barbosa - 2322637
Leticia Lucena - 2322682
Lorena Lin - 2322680
Petrus Galvão - 2322674

DOCUMENTAÇÃO TÉCNICA

Dental Connect

Este documento contém a documentação técnica do Aplicativo Dental Connect de Conexão entre Pacientes e Dentistas desenvolvido na componente curricular N393 - Projeto Aplicado Multiplataforma como requisito para obtenção de nota.

Supervisor: Prof. Bruno Lopes, Me

FORTALEZA

2025

SUMÁRIO

1. INTRODUÇÃO.....	4
1.1. CONTEXTO E JUSTIFICATIVA.....	4
1.2. OBJETIVOS.....	4
1.3. ESCOPO E DELIMITAÇÃO.....	5
2. ENGENHARIA DE REQUISITOS.....	6
2.1. REQUISITOS FUNCIONAIS (RFs).....	6
2.2. REQUISITOS NÃO FUNCIONAIS (RNFs).....	6
3. PROJETO E ARQUITETURA DO SOFTWARE.....	8
3.1. ARQUITETURA GERAL.....	8
3.2. PROJETO DO BANCO DE DADOS.....	8
3.3. PROJETO DE API.....	9
4. TECNOLOGIAS E FERRAMENTAS.....	11
4.1. STACK DE TECNOLOGIAS.....	11
4.2. FERRAMENTAS DE DESENVOLVIMENTO.....	11
5. IMPLEMENTAÇÃO E RESULTADOS.....	13
5.1. TELAS DO SISTEMA.....	13
6. AMBIENTE E GUIA DE IMPLANTAÇÃO.....	14
6.1. REQUISITOS DO AMBIENTE.....	14
6.2. PROCESSO DE IMPLANTAÇÃO.....	14
6.3. ACESSO À APLICAÇÃO IMPLANTADA.....	15
7. CONCLUSÃO.....	16
7.1. TRABALHOS FUTUROS.....	16
7.2. LIÇÕES APRENDIDAS.....	16

1. INTRODUÇÃO

1.1. CONTEXTO E JUSTIFICATIVA

O setor de serviços de saúde, especialmente o odontológico, enfrenta um desafio significativo quando se trata de emergências fora do horário comercial. Pacientes que sofrem com dores agudas, fraturas dentárias ou outros problemas urgentes após o expediente comercial, nos finais de semana ou feriados, muitas vezes encontram dificuldades para localizar um dentista disponível.

O público-alvo principal inclui pessoas que buscam atendimento imediato, mas também abrange dentistas que desejam expandir sua atuação para além do horário tradicional, gerando uma fonte de renda extra e oferecendo um serviço essencial à comunidade. A falta de uma plataforma centralizada para essa conexão resulta em pacientes sofrendo desnecessariamente e profissionais perdendo a oportunidade de atender a uma demanda real e constante.

O aplicativo Dental Connect surge como a solução para essa problemática. Ele existe para preencher essa lacuna, proporcionando uma ponte direta e eficiente entre pacientes em situações de emergência e dentistas dispostos a atendê-los, garantindo que o cuidado odontológico de qualidade esteja acessível a qualquer momento, e não apenas de segunda a sexta, das 9h às 18h.

1.2. OBJETIVOS

O objetivo deste projeto é desenvolver um aplicativo móvel para conectar pacientes e dentistas, facilitando a busca e agendamento de consultas, com foco principal em atendimentos de emergência fora do horário comercial.

Dito isso, os objetivos específicos do projeto são:

- Criar um fluxo de cadastro e autenticação para pacientes e dentistas, garantindo acesso seguro e personalizado.
- Implementar um sistema de busca avançada que permita aos pacientes encontrar dentistas disponíveis de acordo com a geolocalização, especialidade e tipo de tratamento.

- Desenvolver módulos de gerenciamento de consultas para que pacientes possam agendar, visualizar e gerenciar suas consultas de forma eficiente.
- Configurar as páginas de conteúdo para fornecer informações detalhadas sobre os serviços e procedimentos odontológicos oferecidos.
- Estruturar a navegação principal do aplicativo de forma intuitiva, permitindo que os usuários acessem as funcionalidades de forma rápida e simples.
- Integrar um canal de suporte ao usuário para resolver dúvidas e receber feedback.

1.3. ESCOPO E DELIMITAÇÃO

- **Escopo:**
 - **Cadastro e Autenticação:** O sistema permitirá o cadastro e login seguro para dois tipos de usuários: pacientes e dentistas.
 - **Busca Avançada de Profissionais:** Os pacientes poderão buscar dentistas disponíveis usando filtros de geolocalização, especialidade e tipo de tratamento.
 - **Foco em Emergências:** A funcionalidade principal é conectar pacientes a dentistas para atendimentos de emergência fora do horário comercial.
 - **Gerenciamento de Consultas (Paciente):** Os pacientes terão um módulo para agendar, visualizar e gerenciar suas consultas.
 - **Conteúdo Informativo:** O aplicativo terá páginas com informações sobre serviços e procedimentos odontológicos.
 - **Navegação Intuitiva:** A estrutura do aplicativo permitirá acesso rápido às funcionalidades.
 - **Canal de Suporte:** Haverá um canal integrado para que usuários tirem dúvidas e enviem feedback.
- **Delimitação (Fora do Escopo):**
 - **Módulo Financeiro:** O sistema não processará pagamentos de consultas, nem fará gestão de cobranças ou emissão de notas fiscais..
 - **Prontuários Eletrônicos:** O aplicativo não armazenará o histórico médico detalhado (prontuários) dos pacientes nem emitirá receitas médicas.

- **Gestão de Agenda para Dentistas:** O sistema não funcionará como um software completo de gestão de clínica para o dentista (ex: bloqueio de agenda pessoal, controle de estoque, etc.).

2. ENGENHARIA DE REQUISITOS

2.1. REQUISITOS FUNCIONAIS (RFs)

ID	Nome do Requisito	Descrição
RF01	Cadastro de Paciente	O sistema deve permitir que um novo usuário se cadastre como "Paciente", fornecendo nome, e-mail e senha.
RF02	Cadastro de Dentista	O sistema deve permitir que um novo usuário se cadastre como "Dentista", fornecendo dados pessoais e profissionais (ex: CRO).
RF03	Login de Usuário	O sistema deve permitir que usuários façam login usando e-mail e senha.
RF04	Recuperação de Senha	O sistema deve fornecer um fluxo para que usuários possam redefinir suas senhas.
RF05	Gerenciamento de Perfil.	O paciente deve poder visualizar e editar seus dados básicos de cadastro.
RF06	Busca Avançada de Dentistas	O paciente deve poder buscar dentistas disponíveis usando filtros de geolocalização, especialidade e tipo de tratamento.
RF07	Visualização de Perfil do Dentista	O paciente deve poder tocar em um dentista nos resultados da busca para ver seu perfil detalhado.
RF08	Agendamento de Consulta	O paciente deve poder selecionar um horário na agenda disponível do dentista e solicitar um agendamento.
RF09	Visualização de Consultas	O paciente deve ter uma tela para visualizar suas consultas agendadas.
RF10	Conteúdo Informativo	O aplicativo deve exibir telas com informações sobre serviços e procedimentos odontológicos.
RF11	Canal de Suporte	O sistema deve fornecer um formulário ou canal de contato para que usuários enviem dúvidas e feedback.

2.2. REQUISITOS NÃO FUNCIONAIS (RNFs)

- **Desempenho:**

- RNF01: O tempo de inicialização do aplicativo (cold start) não deve exceder 3 segundos, incluindo a exibição da tela de splash configurada para 2 segundos.
- RNF02: A navegação entre telas deve ser fluida, utilizando React Navigation com animações nativas (useNativeDriver: true) para garantir mínimo de 60 frames por segundo durante transições.
- RNF03: O aplicativo deve otimizar o consumo de dados móveis, utilizando estados de loading durante requisições HTTP e evitando requisições desnecessárias ao backend.
- RNF04: As requisições à API devem ter timeout configurado e tratamento de erros adequado para evitar travamentos durante carregamento de dados de clínicas, dentistas e consultas.
- **Usabilidade:**
 - RNF05: A interface deve seguir padrões de design mobile responsivo, utilizando componentes nativos do React Native e bibliotecas como React Native Vector Icons para garantir consistência visual.
 - RNF06: O aplicativo deve fornecer feedback visual imediato ao usuário através de estados de loading, mensagens de erro e animações de feedback durante interações (como seleção de cidade e busca de dentistas).
 - RNF07: A navegação deve ser intuitiva, utilizando Stack Navigator do React Navigation para controle do fluxo de telas, garantindo experiência de usuário familiar e previsível.
- **Compatibilidade:**
 - RNF08: O aplicativo deve ser totalmente funcional em dispositivos Android e iOS, utilizando React Native 0.81.4 e Expo SDK 54, garantindo compatibilidade multiplataforma.
 - RNF09: O aplicativo deve suportar orientação portrait, conforme configurado no app.json, garantindo experiência consistente em diferentes tamanhos de tela.
- **Segurança:**
 - RNF10: O token de autenticação JWT do usuário deve ser gerenciado de forma segura através do contexto de autenticação, sendo transmitido via header Authorization em todas as requisições protegidas à API.
 - RNF11: As senhas dos usuários devem ser criptografadas utilizando bcryptjs no backend antes de serem armazenadas no banco de dados Supabase.
 - RNF12: Todas as requisições HTTP devem utilizar HTTPS quando em produção, garantindo comunicação segura entre o aplicativo e o backend.

- RNF13: O aplicativo deve validar tokens de autenticação em rotas protegidas, retornando erro 403 quando o acesso não for autorizado, conforme implementado no middleware de autenticação.

- **Confiabilidade:**

- RNF14: O aplicativo deve tratar adequadamente erros de rede, exibindo mensagens claras ao usuário quando não for possível conectar ao servidor backend.
- RNF15: O aplicativo deve manter a sessão do usuário durante o uso, utilizando React Context API para gerenciamento de estado global de autenticação.

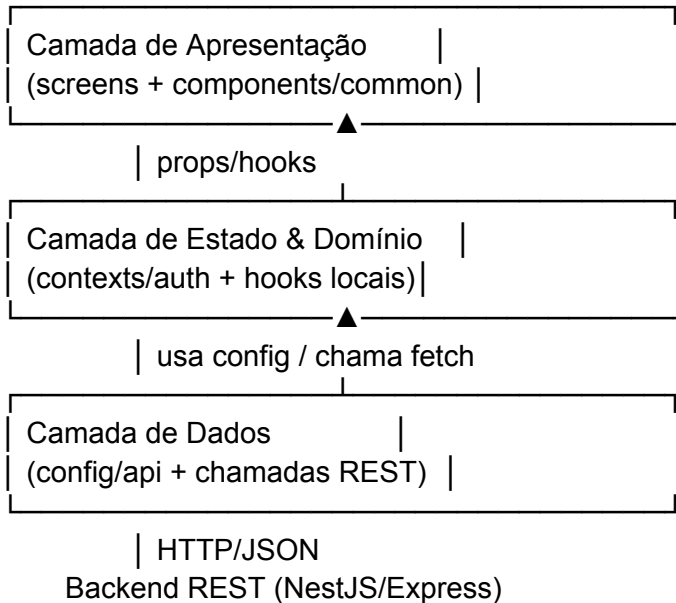
- **Manutenibilidade:**

- RNF16: O código deve seguir padrões de organização modular, separando responsabilidades entre controllers, services, repositories e entities no backend, facilitando manutenção e evolução do sistema.

3. PROJETO E ARQUITETURA DO SOFTWARE

3.1. ARQUITETURA GERAL

O aplicativo DentalConnect adota uma **arquitetura em camadas voltada a componentes** (Presentation → State/Domain → Data), típica de projetos React Native com Expo. Esse arranjo favorece separação de responsabilidades, reuso de UI e centralização da lógica de negócio em contextos e serviços.



Camada de Apresentação (View)

- Composta por `src/screens` e os componentes reutilizáveis em `src/components/common`.
- Cada tela controla apenas estado de UI (por exemplo, `LoginScreen.jsx` guarda campos de email/senha) e delega regras de negócio para o contexto e serviços.
- Navegação entre fluxos (auth, consulta, home) fica isolada em subpastas, evitando colisão de responsabilidades.

Camada de Estado & Domínio (ViewModel/Context)

- Centralizada no `AuthProvider` (`src/context/auth.jsx`), que mantém usuário autenticado, token e ações (`signIn`, `signOut`, etc.).
- O contexto expõe hooks (`useAuth`) consumidos pelas telas, garantindo que a apresentação não conheça detalhes da API nem manipule tokens diretamente.
- Estados específicos de cada fluxo continuam locais aos componentes, mantendo o contexto enxuto e focado em autenticação.

Camada de Dados (Model/Repository)

- Configurações de acesso remoto vivem em `src/config/api.js`, que detecta o host automaticamente no Expo e define todos os endpoints.

- As telas consomem essa configuração para realizar `fetch` direto aos serviços REST. A resposta é validada, os dados relevantes entram no contexto e o restante é mantido localmente.
- Próximos incrementos previstos incluem mover as chamadas para uma pasta `services/` com Axios + interceptors e introduzir persistência segura (ex.: `expo-secure-store`) para o token, consolidando o repositório como “fonte única da verdade”.

Benefícios

- **Separação clara** entre UI, estado e dados reduz acoplamento e facilita testes unitários do contexto.
- **Reuso de componentes** (inputs, botões) padroniza a experiência visual.
- **Configuração única de API** simplifica a troca de ambientes e evita strings duplicadas.

3.2. PROJETO DE DADOS: INTEGRAÇÃO COM API E PERSISTÊNCIA LOCAL

O DentalConnect consome uma **API REST** como fonte primária de informação e repassa os dados às telas via hooks/contexts. Neste estágio do projeto não há cache persistente no dispositivo; o app mantém estados apenas em memória (React Context + estados locais). A seguir detalho o fluxo e a estratégia planejada para suportar experiências offline.

Usuário → Screens → Context/Auth → Fetch (API_CONFIG) → Backend REST
 |
 └─(futuro) cache seguro (SecureStore/SQLite)

Integração com a API

- `src/config/api.js` centraliza `BASE\_URL` e endpoints, detectando automaticamente o host do Expo ou usando fallbacks estáticos. Todas as telas importam essa configuração para montar as URLs de `fetch`.
- Os fluxos principais (login, agendamento, listagem de clínicas/procedimentos) chamam a API diretamente nas screens. A resposta JSON é validada, tratada e propagada aos componentes visuais.
- O contexto de autenticação (`src/context/auth.jsx`) guarda `user` e `token` em memória, permitindo que qualquer tela recupere o estado atual sem refazer login durante a mesma sessão.

Persistência Local

- **Situação atual:** nenhum dado é salvo em `AsyncStorage`, SQLite ou similares. Ao fechar o app, credenciais e dados carregados são descartados. Isso simplifica o protótipo, mas impede funcionamento offline e auto-login.
- **Limitações decorrentes:**
 - Dependência total de rede para autenticação e consultas.
 - Maior latência, pois cada tela precisa solicitar dados novamente ao abrir.
 - Sem “single source of truth” local, o app não consegue exibir o último estado quando a API falha.

Estratégia Evolutiva

- **Camada de serviço dedicada:** mover as chamadas `fetch` para módulos `services/` com Axios + interceptors, concentrando serialização, tratamento de erros e refresh do token.
- **Cache seguro de credenciais:** persistir token e perfil no `expo-secure-store` (ou `AsyncStorage` + Criptografia) para suportar login automático.
- **Cache de dados funcionais:** usar uma combinação de `expo-sqlite`/`react-query` para guardar consultas, clínicas e procedimentos.

O fluxo desejado:

1. A tela solicita dados ao serviço/repositório.
2. O serviço tenta atualizar da API; ao receber resposta válida, sincroniza o cache local.
3. A UI consome diretamente o cache (fonte única da verdade), atualizando assim que novos dados chegam.
4. Em caso de falha de rede, o app mantém o último snapshot local, permitindo navegação de consulta offline.

Essa estratégia assegura melhor performance, resiliência em cenários com internet instável e prepara o aplicativo para requisitos de conformidade (armazenamento cifrado de tokens e dados sensíveis).

Dicionário de Dados (Tabela **Consulta**):

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
id	UUID	PK	Não	Identificador único da consulta, primário.
paciente_id	UUID	FK	Não	Chave estrangeira ligando à tabela de Usuários .
dentista_id	UUID	FK	Não	Chave estrangeira ligando à tabela de Dentistas .
data_hora	TIMESTAMPZ		Não	Data e hora agendada da consulta, com fuso horário.
status	VARCHAR(20)		Não	Situação atual da consulta (Ex: "Agendado", "Confirmado", "Realizado", "Cancelado").
last_updated	TIMESTAMPZ		Não	Timestamp de quando o registro foi sincronizado pela última vez com a API. .

procedimento	VARCHAR(255)		Não	Descrição do procedimento a ser realizado (Ex: "Limpeza", "Restauração").
--------------	--------------	--	-----	---

3.3. CONSUMO DA API E FLUXO DE NAVEGAÇÃO

O aplicativo **DentalConnect** utiliza uma arquitetura de consumo de API híbrida, fazendo uso de um serviço *Backend-as-a-Service (BaaS)* e uma API REST Customizada (Desenvolvimento Próprio) para gerenciar dados e lógica de negócios.

Referências de API Consumidas

O DentalConnect consome dois serviços de API principais:

- **API Supabase (Infraestrutura e Autenticação):**
 - Esta API é utilizada para operações de infraestrutura, incluindo **autenticação de usuários, gerenciamento de banco de dados e armazenamento de arquivos**.
 - A documentação oficial para a biblioteca de cliente JavaScript (utilizada no ambiente React Native) pode ser acessada através do seguinte link: [Documentação Oficial do Supabase Client para JavaScript](#)
- **API REST Customizada (Lógica de Negócios):**
 - Esta API é o backend próprio do projeto, desenvolvida para lidar com as regras de negócio específicas do agendamento de consultas e gerenciamento de dentistas.
 - **Endpoints Chave:** As rotas principais incluem */consultation*, */dentists/nome*, */locals* e */users*, conforme identificado no arquivo de configuração.

O fluxo de navegação do usuário dentro do aplicativo foi projetado para ser simples e intuitivo, conforme ilustrado no diagrama a seguir:

Diagrama de Fluxo de Navegação

O Diagrama de Fluxo de Navegação a seguir ilustra as principais telas do aplicativo DentalConnect e como um usuário (provavelmente um paciente) transita entre elas para alcançar o objetivo de **agendar uma consulta**.

4. TECNOLOGIAS E FERRAMENTAS

4.1. STACK DE TECNOLOGIAS

- **Linguagem:** JavaScript (ES6+), escolhida por sua versatilidade e ampla adoção no desenvolvimento mobile com React Native, oferecendo sintaxe moderna e recursos como arrow functions, destructuring e async/await.
- **Framework Mobile:** React Native, selecionado por permitir desenvolvimento multiplataforma (iOS e Android) com uma única base de código, oferecendo performance nativa e acesso às APIs do dispositivo.
- **Plataforma de Desenvolvimento:** Expo, utilizada para simplificar o processo de desenvolvimento, build e deploy da aplicação React Native, fornecendo ferramentas integradas e serviços de desenvolvimento.
- **Backend:** Node.js com Express.js, escolhido por sua eficiência em I/O assíncrono e vasto ecossistema de pacotes npm, permitindo desenvolvimento rápido de APIs REST.
- **Banco de Dados:** Supabase (PostgreSQL), selecionado como Backend-as-a-Service por oferecer autenticação integrada, banco de dados relacional, storage de arquivos e APIs REST automáticas.
- **Autenticação:** Supabase Auth integrado com JWT (JSON Web Tokens) para gerenciamento seguro de sessões de usuário e controle de acesso.
- **Navegação:** React Navigation, biblioteca oficial para implementação de navegação entre telas, utilizando Stack Navigator para controle do fluxo da aplicação.
- **Gerenciamento de Estado:** React Context API, escolhido para gerenciamento de estado global da aplicação, especialmente para dados de autenticação e informações compartilhadas entre componentes.
- **Persistência Local:** AsyncStorage, utilizada para armazenamento local de dados do usuário e cache offline da aplicação.
- **UI Components:** React Native Vector Icons para ícones customizados e componentes nativos do React Native para construção da interface.
- **Comunicação HTTP:** Fetch API nativo do JavaScript para realizar chamadas HTTP à API REST, com tratamento de promises e async/await.
- **Validação e Segurança:** bcryptjs para hash de senhas e jsonwebtoken para geração e validação de tokens de autenticação.

4.2. FERRAMENTAS DE DESENVOLVIMENTO

- **IDE:** Visual Studio Code foi utilizado como ambiente de desenvolvimento principal, aproveitando suas extensões para React Native, JavaScript/JSX, Git e integração com Expo. A IDE oferece suporte completo para desenvolvimento mobile com debugging integrado.

- **Controle de Versão:** Git foi adotado para controle de versão, com repositório hospedado no GitHub. O projeto utiliza branches separadas para diferentes funcionalidades, incluindo branches específicas como "consulta-por-clinica" para desenvolvimento de features isoladas.
- **Framework Mobile:** Expo CLI foi utilizado para inicialização, desenvolvimento, build e execução do projeto React Native, proporcionando ferramentas integradas para desenvolvimento e deploy.
- **Backend:** Node.js com Express.js foi configurado para desenvolvimento da API REST, utilizando middleware para CORS, parsing de JSON e roteamento de endpoints.
- **Banco de Dados:** Supabase Dashboard utilizado para gerenciamento do banco de dados PostgreSQL, configuração de políticas de segurança e monitoramento de dados.
- **Teste de API:** Durante o desenvolvimento, ferramentas como curl via terminal e testes diretos foram utilizados para validar os endpoints da API e verificar a comunicação entre frontend e backend.
- **Gerenciamento de Dependências:** npm foi utilizado para instalação e gerenciamento de pacotes tanto no frontend quanto no backend, com package.json para controle de versões.
- **Build e Deploy:** Expo Build Service foi utilizado para compilação da aplicação para diferentes plataformas, facilitando o processo de distribuição.

Tela inicial do sistema, com interface limpa e minimalista, plenamente funcional e integrada ao banco de dados. Uso intuitivo, permite a visualização da senha digitada e a seleção da tela que envia um contato para recuperar a conta cuja senha tenha sido esquecida.

20:26

7%

Entre no DentalConnect :)

E-mail

annafgcorreia@gmail.com

Senha

.....



☐ Lembrar meus dados

[Esqueci minha senha](#)

Entrar

[Criar conta](#)



Figuras 3 e 4: Home

Tela exibida após o acesso do usuário com perfil de Paciente. Exibe várias seções divididas por funcionalidade (acesso aos agendamentos, busca por dentistas, lista de procedimentos disponíveis para marcação através do DentalConnect, estatísticas de uso e mais).

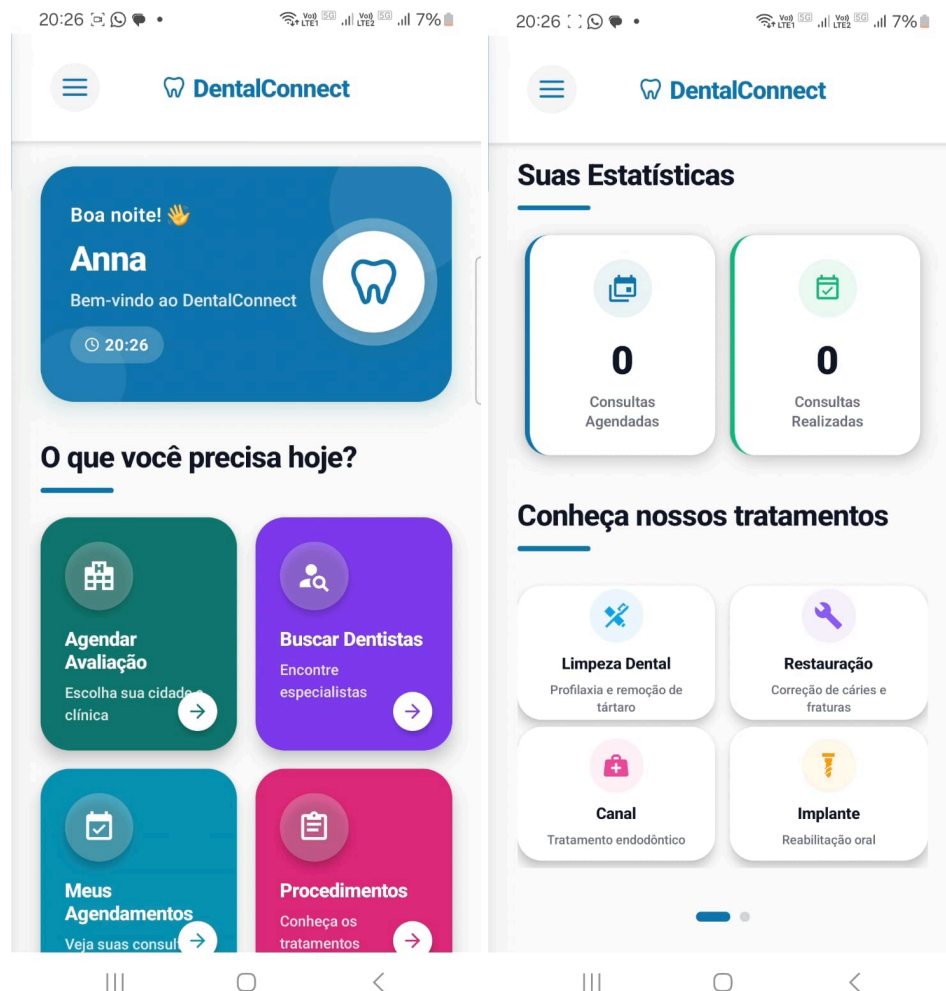
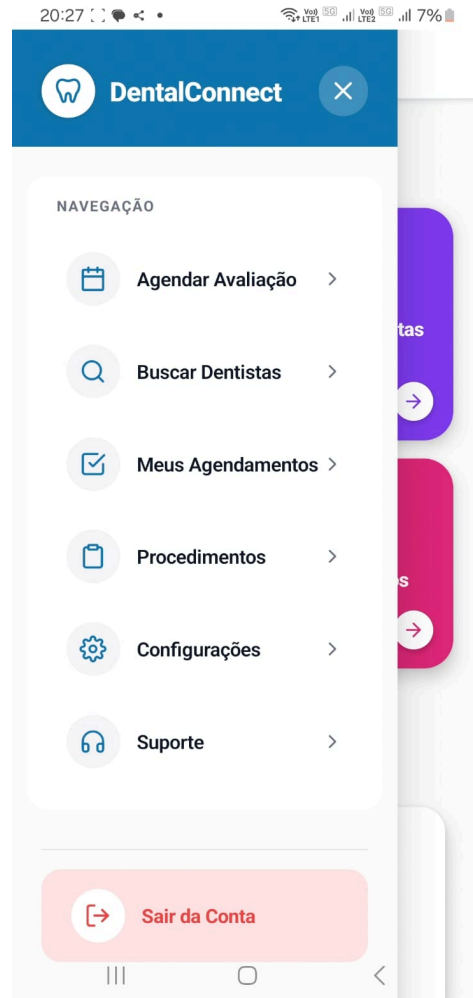


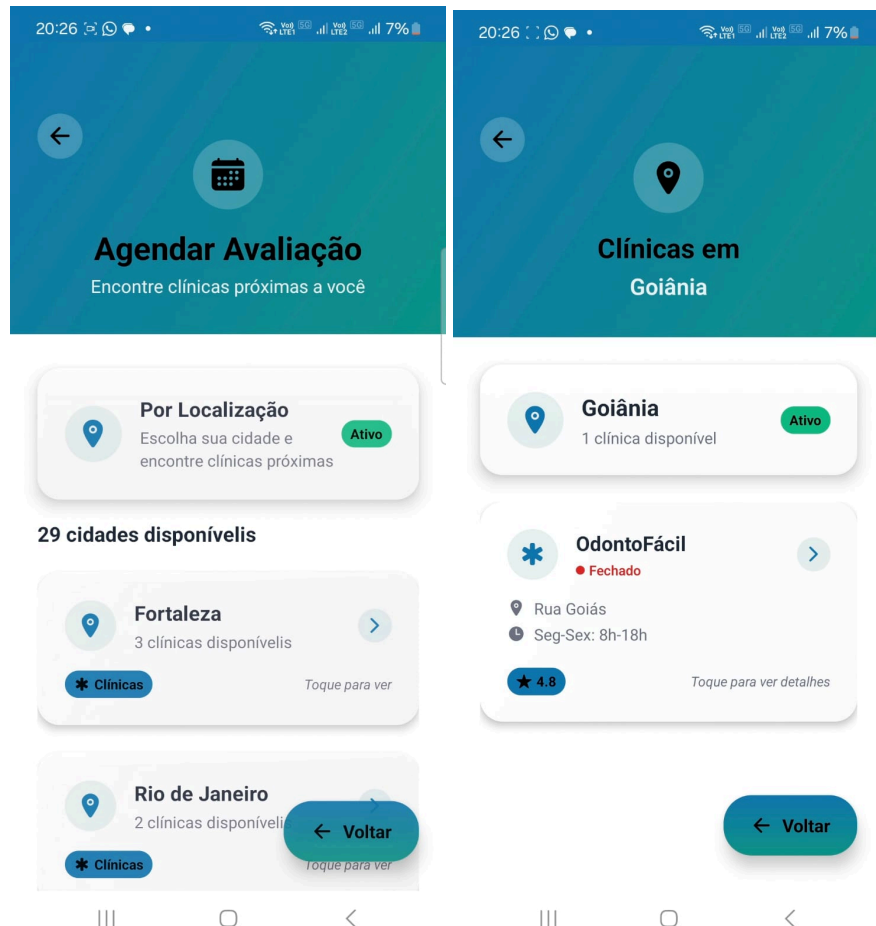
Figura 5: Menu de navegação lateral

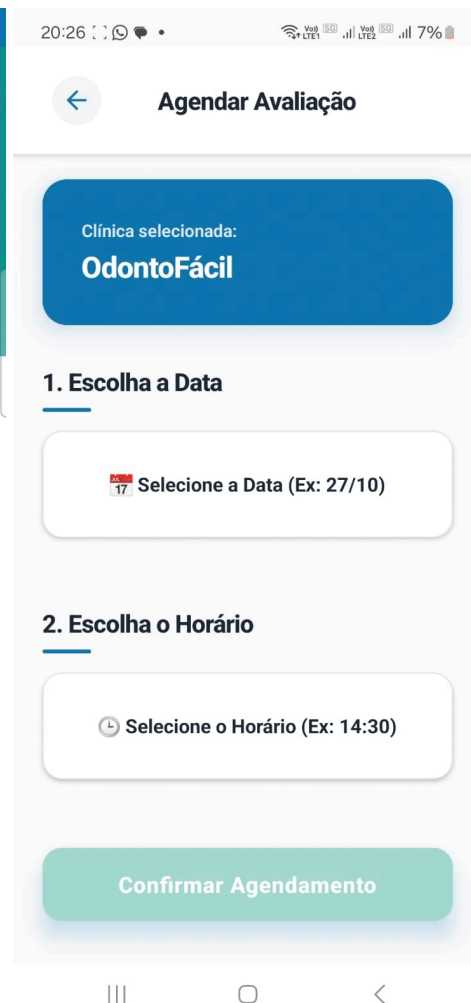
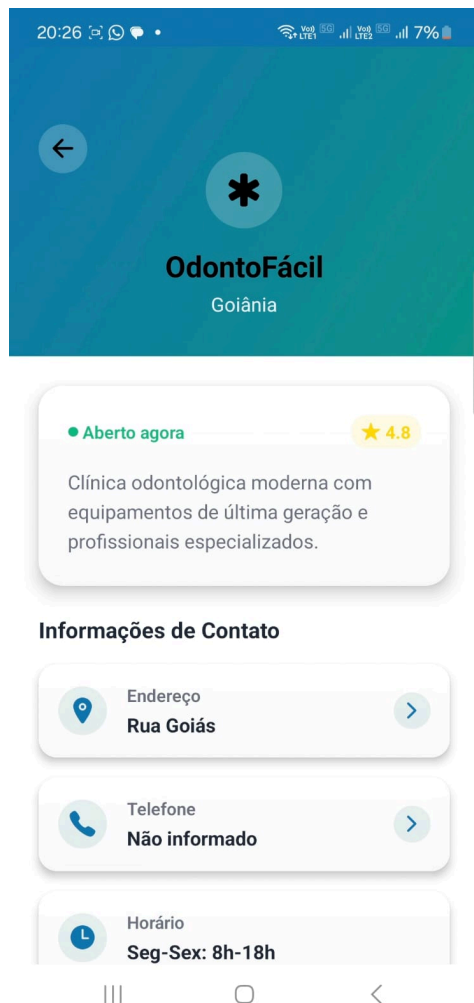
Conforme exibido na imagem abaixo, o menu de navegação lateral está disponível a todo tempo para facilitar e viabilizar o percurso do usuário pelas diferentes funcionalidades e telas do aplicativo.



Figuras 6, 7, 8 e 9: Agendar avaliação

As telas abaixo demonstram o fluxo de uso para agendar uma avaliação, permitindo a busca por localização e a listagem de locais disponíveis. Ao selecionar uma cidade, por exemplo, é possível visualizar os cards das clínicas (Figura 7), ler os detalhes (Figura 8) e abrir a janela para realizar o agendamento (Figura 9).





Tela que permite ao usuário pesquisar um dentista específico por parte do nome, como demonstrado abaixo. O sistema retorna resultados com consultas a partir de uma letra.

Figura 11: Nossos procedimentos

Tela que exhibe ao usuário uma visualização rápida, com descrição, de todos os procedimentos de tratamento dental que podem ser agendados através do DentalConnect. Serve como referência rápida caso surjam dúvidas na hora de realizar os agendamentos.



Figuras 12 e 13: Configurações e Perguntas frequentes

A tela de Configurações (Figura 12) permite ao usuário editar suas informações pessoais, tirar dúvidas rápidas sobre o funcionamento da plataforma (Figura 13) e entrar em contato com o suporte da DentalConnect (Figura 14), além de sair da conta.

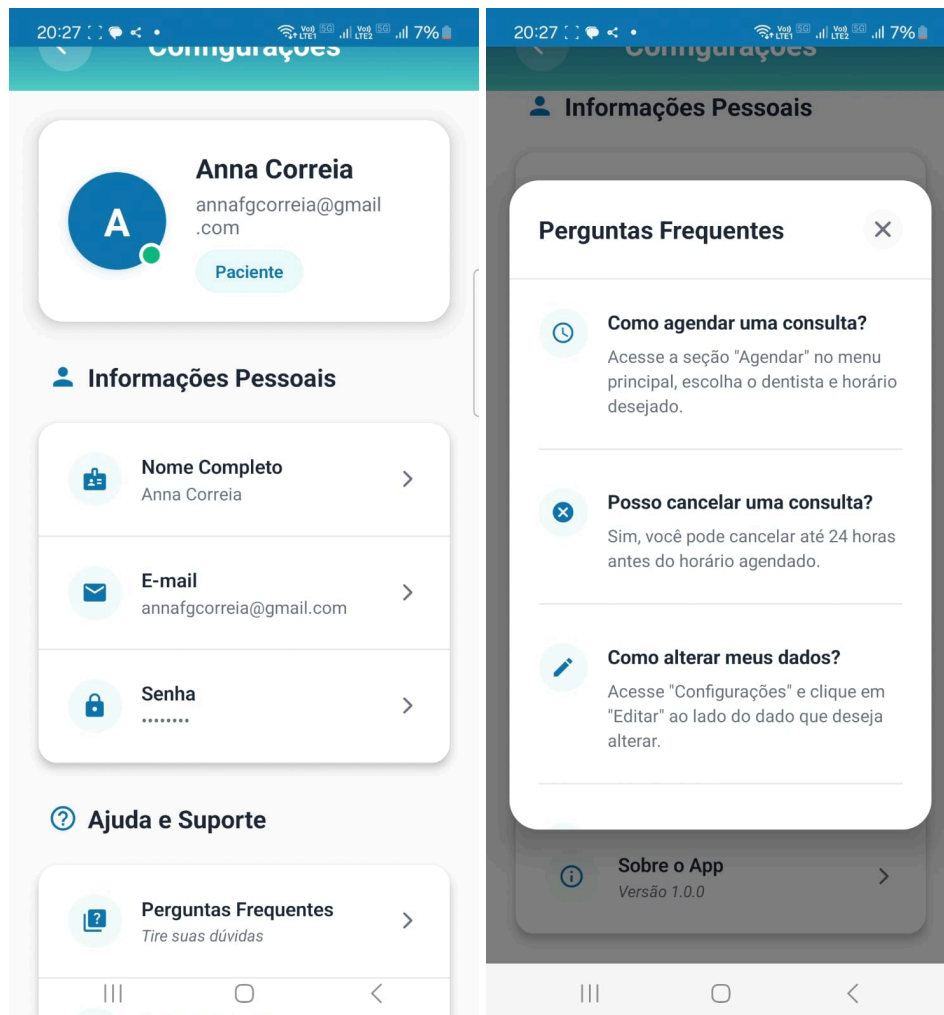


Figura 14: Fale Conosco

A tela Fale Conosco contém um formulário simples e funcional que coleta os dados principais de contato do usuário (caso já não estejam disponíveis no próprio cadastro) e recebe uma comunicação sobre problemas técnicos ou dúvidas não disponíveis na seção adequada, permitindo um acompanhamento rápido para solucionar os problemas.

20:27 7%

← Configurações

Fale Conosco

Tem dúvidas ou precisa de ajuda? Entre em contato conosco.
Vamos fazer o possível para te ajudar. :)

Avenida Santos Dumont, 3060 - Fortaleza/CE
(85) 98765-4321
faleconosco@dentalconnect.com

Envie sua mensagem

Nome

E-mail

Telefone (opcional)

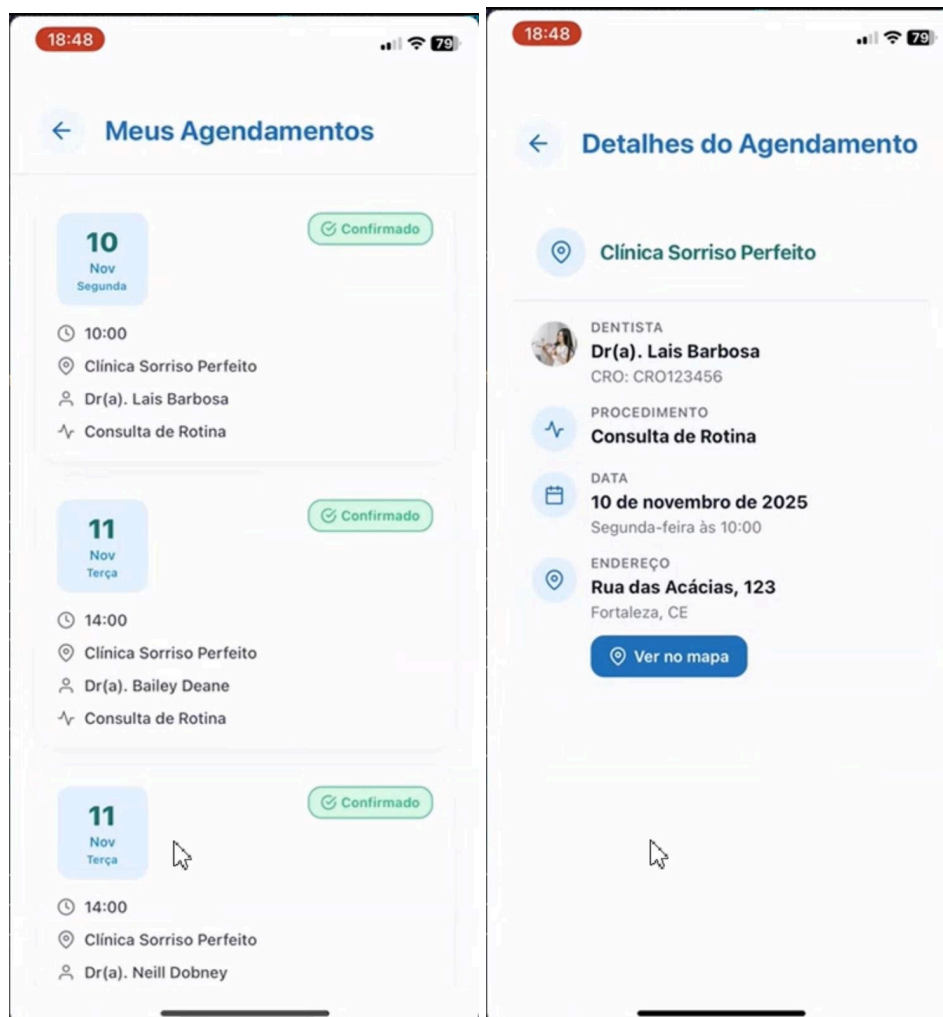
Mensagem

Enviar Mensagem

Figuras 15 e 16: Meus agendamentos

As telas Meus Agendamentos permitem ao usuário do tipo Paciente acompanhar o status dos horários programados para consultas e demais atendimentos marcados através do DentalConnect (Figura 15). É possível acessar uma visão rápida do horário, local, profissional e tipo de serviço selecionado, além do status da solicitação.

Na tela Detalhes do Agendamento (Figura 16), o paciente pode visualizar as informações aprofundadas como o número do CRO do profissional, o endereço da clínica e um botão integrado com aplicativo de georreferenciamento.



6. AMBIENTE E GUIA DE GERAÇÃO (BUILD)

Para compilar o código-fonte e gerar o arquivo APK da plataforma DentalConnect, é necessário configurar o ambiente conforme as especificações indicadas abaixo:

6.1. REQUISITOS DO AMBIENTE

- **IDE:** Visual Studio Code (versão mais recente recomendada)
- **Node.js:** Versão 18.x ou superior (LTS recomendada)
- **npm:** Versão 9.x ou superior (incluído com Node.js)
- **Expo CLI:** Versão 6.x ou superior
- **Git:** Versão 2.x ou superior para controle de versão
- **Sistema Operacional:** Windows 10/11, macOS ou Linux

Também é necessário garantir que as principais dependências e bibliotecas do projeto estejam instaladas, seguindo o processo descrito na seção 6.2.

A versão Mobile do DentalConnect está sendo desenvolvida com React Native 0.81.4 e React 19.1.0, com Expo SDK 54. Entre as bibliotecas principais, estão:

- **react-navigation/native** e **react-navigation/stack**, para controle de navegação entre telas;
- **react-native-async-storage/async-storage**, para armazenamento local de dados;
- **supabase/supabase-js**, para integração com o backend Supabase (autenticação, banco de dados e APIs).
- **axios**, cliente HTTP para consumo de APIs.
- **react-native-calendars**, componente de calendário e agendamentos.
- **expo-linear-gradient**, **expo-status-bar** e **expo/vector-icons**, recursos visuais e ícones nativos do Expo.

6.2. PROCESSO DE GERAÇÃO DE APLICATIVO

1. Clone o repositório do projeto front-end: `git clone https://github.com/linlorena/dentalconnect-mobile.git`
2. Clone o repositório do projeto back-end: `git clone https://github.com/laissilva04/dentalconnect-backend.git`
3. Configure o ambiente do front-end:

```
cd ../dentalconnect-mobile/frontend
npm install
npx expo start
```
4. Inicie o servidor backend:

```
cd ../../dentalconnect-backend
npm start
```

OU

```
npx nodemon index.js
```

5. Teste o funcionamento do servidor backend na porta definida.

```
localhost:3000
```

6. Gere a versão instalável do APK:

```
npx expo prebuild
```

```
npx expo build:android
```

OU

```
npx eas build -p android --profile preview
```

6.3. ACESSO À APLICAÇÃO IMPLANTADA

Instrução: Forneça um link para o download direto do arquivo .apk ou para uma plataforma de testes (como Firebase App Distribution) onde a banca possa instalar o aplicativo.

Exemplo de Texto:

- **Link para Download do APK:** [Link do Google Drive/Dropbox para o arquivo .apk]
- **Credenciais de Acesso (Perfil de Vendedor):**
 - **Usuário:** vendedor@teste.com
 - **Senha:** vendedor123

7. CONCLUSÃO

7.1. TRABALHOS FUTUROS

Conforme apresentado neste documento, a plataforma DentalConnect surge como uma solução essencial para atender uma necessidade com a qual vários usuários já se depararam ao longo da vida: conectar pacientes em situações emergenciais a profissionais dispostos a atendê-los fora do horário comercial convencional. Com uma interface simples e intuitiva, o DentalConnect oferece uma usabilidade fácil e praticidade em momentos complicados, capaz de oferecer rapidez e atendimento acessível.

Entretanto, reconhecemos a necessidade de implementar melhorias contínuas e aprimorar o aplicativo para agregar valor e fidelizar o usuário, bem como expandir sua reputação dentro do mercado e base de clientes, tanto para usuários pacientes quanto para profissionais da odontologia. Dessa forma, listamos abaixo alguns pontos que podem ser explorados num futuro próximo:

- **Versão para iOS:**
Desenvolver uma versão compatível com iOS possibilitaria a expansão da base de usuários, adentrando outros segmentos de mercado.
- **Sistema interno de avaliação:**
Criar um sistema de avaliação que permita aos pacientes avaliar o atendimento recebido, fortalecendo a confiança no trabalho dos dentistas e da plataforma.
- **Comunicação integrada por chat e chamadas:**
Implementar uma funcionalidade de teleconsultas rápidas por áudio e/ou vídeo, permitindo uma avaliação prévia do paciente antes do deslocamento. O chat também serviria para tirar dúvidas após o atendimento ou contatos imediatos.
- **Mais funcionalidades georreferenciadas:**
Evoluir o uso de georreferenciamento para sugerir automaticamente profissionais próximos e calcular o tempo estimado e o melhor meio de deslocamento até a clínica.

7.2. LIÇÕES APRENDIDAS

O desenvolvimento da versão mobile do DentalConnect proporcionou um conjunto valioso de aprendizados técnicos, organizacionais e colaborativos. O processo foi desafiador por tratar de modificações e adaptações com base na estrutura que havia sido desenvolvida previamente para a versão Desktop, sendo necessário aprimorar-se no desenvolvimento em ReactNative aos poucos. A comunicação,

resolução de dúvidas e o trabalho de equipe, principalmente, foram fundamentais para garantir que esta empreitada fosse concluída.

Nossos aprendizados principais podem ser elencados nas temáticas abaixo:

1. Aprendizado em plataformas mobile

A maior dificuldade encontrada foi garantir uma experiência fluida durante a busca e atualização das informações de profissionais, especialmente ao lidar com filtros aplicados. A diferença em relação à versão Desktop desenvolvida anteriormente foi um ponto de atenção ao modelar a estrutura do aplicativo, e o gerenciamento de estado também exigiu atenção para que a interface refletisse corretamente os dados puxados da API. Este ponto reforçou a necessidade de considerar a estrutura e escalabilidade do trabalho desde o início para prevenir retrabalhos e rupturas no futuro.

2. Organização e priorização de demandas

Tornou-se evidente para todos os integrantes da equipe que organização, comunicação e divisão equilibrada de tarefas eram necessidades de primeira ordem para que o desenvolvimento seguisse conforme o planejado. Entender o escopo e priorizar o desenvolvimento de telas e funcionalidades na ordem correta evitou retrabalhos e permitiu que cada integrante do grupo evoluísse tecnicamente.

Acima de tudo, o aprendizado principal do desenvolvimento do DentalConnect é a percepção de que fazer um aplicativo simples, eficiente e funcional não diz respeito apenas ao código, mas também à clareza arquitetural, empatia e compreensão das necessidades do paciente e do profissional de saúde e também da equipe envolvida, que deu o melhor de si apesar de rotinas exigentes e questões externas ao ambiente de trabalho. O caminho trilhado ao longo deste semestre e dos melhoramentos adotados no DentalConnect serviu não somente para o aprimoramento profissional dos membros da equipe, mas também para o reforço de laços de companheirismo.